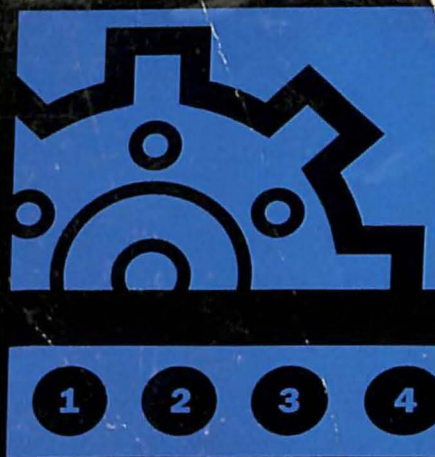
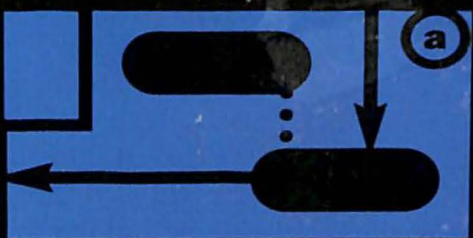
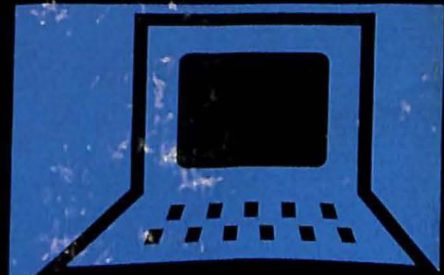
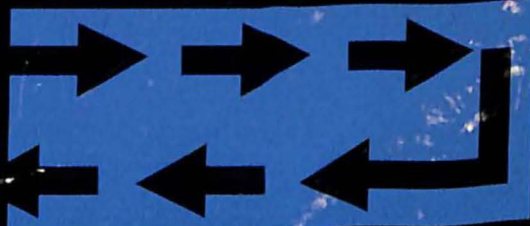


$$\Gamma_L(f) = \frac{Z_L(f)}{Z(f)}$$



Programación en Linux

C O N E J E M P L O S



```
printf("\n\n\n\n\nWhat is the  
name? "); gets(item.name);  
printf("What is the city?  
"); gets(item.city);  
getzip(item.zipcode); ans = g  
char(); if((fp=fopen(FILENAME,  
"r+"))==NULL){ err_msg("R  
Read error-ensure file retu
```



Prentice
Hall

que®

Kurt Wall

KURT WALL

Programación en Linux

C O N E J E M P L O S



**BIBLIOTECA
UNIVERSIDAD de PALERMO**

Prohibida su Reproducción - Ley 11723



**Argentina • Bolivia • Brasil • Colombia • Costa Rica • Chile • Ecuador • Salvador •
España • Guatemala • Honduras • México • Nicaragua • Panamá • Paraguay • Perú •
Puerto Rico • República Dominicana • Uruguay • Venezuela**

**Amsterdam • Harlow • Londres • Menlo Park • Miami • Munich • Nueva Delhi • Nueva Jersey •
Nueva York • Ontario • París • Singapur • Sydney • Tokio • Toronto • Zurich**

datos de catalogación bibliográfica

519.68
WAL

Wall, Kurt

Programación en Linux con ejemplos - 1ª ed -

Buenos Aires: Prentice Hall, 2000

568 p.; 24x19 cm

Traducción de: Jorge Gorín

ISBN: 987-9460-09-X

I. Título - 1. Programación

UNIVERSIDAD DE PALERMO
BIBLIOTECA

042613



Fecha recepción: 08.10.03

Procedencia: COMPRA

Editora: María Fernanda Castillo
Armado de interior y tapa: Pérez Villamil & Asociados
Traducción: Jorge Gorín
Producción: Marcela Mangarelli

Traducido de:
Linux Programming by example, by Kurt Wall, published by QUE.
Copyright © 2000,
All Rights Reserved.
Published by arrangement with the original publisher,
PRENTICE HALL, inc, A Pearson Education Company.
ISBN: 0-7897-2215-1

Edición en Español publicada por Pearson Education, S.A.
Copyright © 2000
ISBN: 987-9460-09-X

Este libro no puede ser reproducido total ni parcialmente en ninguna forma, ni por ningún medio o procedimiento, sea reprográfico, fotocopia, microfilmación, mimeográfico o cualquier otro sistema mecánico, fotoquímico, electrónico, informático, magnético, electroóptico, etcétera. Cualquier reproducción sin el permiso previo por escrito de la editorial viola derechos reservados, es ilegal y constituye un delito.

© 2000, PEARSON EDUCATION S.A.
Av. Regimiento de Patricios 1959 (1266), Buenos Aires,
República Argentina

Queda hecho el depósito que dispone la ley 11.723
Impreso en Perú. Printed in Perú.
Impreso en Quebecor Perú, en el mes de noviembre de 2000
Primera edición: Diciembre de 2000

Associate Publisher
Dean Miller

Executive Editor
Jeff Koch

Acquisitions Editor
Gretchen Ganser

Development Editor
Sean Dixon

Managing Editor
Lisa Wilson

Project Editor
Tonya Simpsons

Copy Editor
Kezia Endsley

Indexer
Cheryl Landes

Proofreader
Benjamin Berg

Technical Editor
Cameron Laird

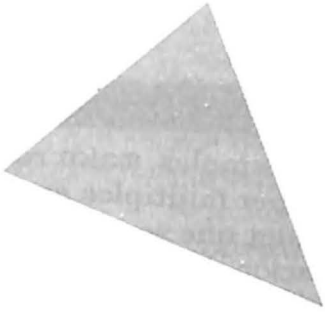
Team Coordinator
Cindy Teeters

Interior Design
Karen Ruggles

Cover Design
Duane Rader

Copy Writer
Eric Borgert

Production
Dan Harris
Heather Moseman



Control del proceso de compilación: el make de GNU

`make` es una herramienta utilizada para controlar los procesos de construcción y revisión de software. `make` automatiza el tipo de software que se crea, cómo y cuándo se lo desarrolla, permitiendo al programador concentrarse exclusivamente en la redacción del código fuente. Además permite ahorrar muchísimo tipeo, ya que contiene decisiones lógicas integradas que invocan el compilador de C con las opciones y argumentos que sean adecuados.

Este capítulo cubre los siguientes temas:

- Cómo invocar a `make`
- Las opciones de línea de comando y los argumentos de `make`
- Creación de `makefiles`
- Redacción de reglas para `make`
- Utilización de las variables de `make`
- Administración de errores en `make`

Todos los programas de este capítulo pueden ser encontrados en el sitio Web <http://www.mcp.com/info> bajo el número de ISBN 0789722151.

¿Qué uso tiene make?

Para todos los proyectos de software excepto los más simples, `make` resulta esencial. En primer lugar, los proyectos compuestos por múltiples archivos de código fuente requieren invocaciones del compilador que son largas y complejas. `make` simplifica esto mediante la invocación de estas difíciles líneas de comandos en un `makefile`, un archivo de texto que contiene todos los comandos requeridos para desarrollar proyectos de software.

`make` resulta conveniente para todo aquel que quiera desarrollar un programa. A medida que se van haciendo modificaciones al programa, ya sea que se le agreguen nuevas prestaciones o se incorporen correcciones a errores (bugs) detectados, `make` le permite a uno reconstruir el programa con un único y breve comando, reconstruir sólo un único módulo, tal como un archivo de biblioteca, o personalizar el tipo de desarrollo que se desee, según las circunstancias. `make` resulta también conveniente para otra gente que podría desarrollar el programa. En lugar de crear documentación que explique en penoso detalle cómo desarrollarlo, uno simplemente le debe indicar que tipeen `make`. El programador original apreciará las ventajas de tener que escribir menor cantidad de documentación, y los demás colaboradores apreciarán la conveniencia de comandos de desarrollo que sean simples.

✓ Para obtener mayor información acerca de la distribución de software, ver “Distribución de software”, página 445.

Finalmente, `make` acelera el proceso de edición-compilación-depuración. Minimiza los tiempos de recompilación porque es lo suficientemente inteligente como para determinar qué archivos han cambiado, y por lo tanto sólo reconstruye los archivos cuyos componentes han cambiado. El `makefile` también constituye una base de datos de información sobre *dependencias* entre archivos para los proyectos de un programador, permitiéndole verificar automáticamente que todos los archivos necesarios para compilar un programa estén disponibles cada vez que uno inicia una compilación. A medida que el lector vaya ganando experiencia con la programación en Linux, apreciará esta característica de `make`.

Utilización de make

Esta parte explica cómo utilizar `make`. En particular, explica cómo crear `makefiles`, cómo invocar a `make`, cómo desmenuzar su plétora de opciones, argumentos y especificadores, y cómo administrar los inevitables errores que ocurren por el camino.

Creación de makefiles

Así que, ¿cómo logra realizar make sus mágicas proezas? Utilizando un makefile que es una base de datos en formato texto que contiene reglas que le indican a make qué construir y cómo construirlo. Cada regla consiste de lo siguiente:

- Un target, la “cosa” que en definitiva make trata de crear.
- Una lista de una o más *dependencias*, generalmente archivos, requeridas para construir el target.
- Una lista de comandos a ser ejecutados con el fin de crear el target a partir de las *dependencias* entre archivos especificadas.

Cuando se lo invoca, el make de GNU busca un archivo denominado GNUmakefile, makefile, o Makefile, en ese orden. Por alguna razón, la mayoría de los programadores de Linux utiliza esta última forma, Makefile.

Las reglas de un makefile tienen la siguiente forma general:

```
objetivo : dependencia [dependencia] [...]
        comando
        [comando]
        [...]
```

PRECAUCIÓN

El primer carácter de un comando debe ser el carácter de tabulación; ocho espacios no son la misma cosa. Esto a menudo toma a la gente desprevenida, y puede resultar un problema si su editor preferido convierte “servicialmente” cada tabulador en ocho espacios. Si uno trata de emplear espacios en lugar de tabuladores, make exhibirá el mensaje “Missing separator” y se detendrá.

Target es habitualmente el archivo binario o el archivo objeto que uno quiere crear. *Dependencia* es una lista de uno o más archivos requeridos como fuente con el fin de crear target. Los comandos son los diversos pasos, tales como la invocación del compilador o los comandos de interfaz, necesarias para crear el target. A menos que se lo especifique de forma expresa, make realiza todo su trabajo en el directorio corriente de trabajo.

“Todo muy bien”, debe estar pensando probablemente el lector, “¿pero cómo logra saber make cuándo reconstruir un archivo?” La respuesta es sorprendentemente simple: si un target especificado no se encuentra presente en algún lugar donde make pueda hallarlo, make lo (re)construye. Si el target no existe, make compara la fecha y hora del objetivo (target) con la fecha y hora de los archivos de los cuales éste depende. Si al menos una de dichos archivos es más nueva que el target, make reconstruye el target, interpretando que el hecho de que ese archivo sea más nuevo es señal de que debe incorporarse al mismo algún cambio de código.



Ejemplo

Este makefile de muestra permitirá que la explicación sea más concreta. Sería el makefile necesario para desarrollar un editor de textos al que, haciendo gala de imaginación, hemos denominado editor. Los signos de numeral (#) inician las líneas de comentarios, tal como se comenta en la página 50. gcc es el comando que activa el compilador. rm (ver tabla 2.3, página 47) es una variable predefinida que activa un programa de eliminación de archivos.

```
#
# Makefile de muestra
#
editor : editor.o pantalla.o teclado.o
    gcc -o editor editor.o pantalla.o teclado.o
editor.o : editor.c editor.h teclado.h pantalla.h
    gcc -c editor.c
pantalla.o : pantalla.c pantalla.h
    gcc -c pantalla.c
teclado.o : teclado.c teclado.h
    gcc -c teclado.c
prolijar :
    rm editor *.o
```

Para compilar editor, se debería tipear sencillamente make en el directorio donde se encuentre el makefile. Así de simple.

Este makefile tiene cinco reglas. El primer target, editor, se denomina *target predeterminado*; este es el archivo que make en definitiva trata de crear. editor tiene tres archivos de los cuales depende: editor.o, pantalla.o y teclado.o; estos tres archivos deben existir para que se pueda construir editor. La línea siguiente consiste en el comando que debe ejecutar make para crear editor. Como se recordará del capítulo 1, “Compilación de programas”, esta invocación del compilador construye un archivo ejecutable denominado editor a partir de los tres módulos objeto, editor.o, pantalla.o y teclado.o. las tres reglas que vienen después le indican a make cómo construir cada uno de los módulos objeto. La última regla le indica a make que haga limpieza, eliminando todos los archivos objeto que contribuyen a formar editor.

Aquí es donde el valor de make se hace evidente: normalmente, si uno tratase de construir editor utilizando el comando de compilador de la línea 2, si los archivos de los cuales éste dependiera no existiesen gcc produciría un tajante mensaje de error y terminaría. make, en cambio, después de haber determinado que editor requiere esos archivos para ser compilado, primero verificaría que existiesen y, si no fuera así, ejecutaría los comandos necesarios para crearlos. Luego retornaría a la primera regla y crearía del archivo ejecutable de editor. Por supuesto, si a su vez las respectivas dependencias para los componentes, teclado.c o pantalla.h, no existiesen, make renunciaría también, porque no dispondría de los targets denominados, en este caso, teclado.c y pantalla.h.

Invocación de make

Invocar a make sólo requiere tipear make en el directorio donde se encuentre el makefile. Por supuesto, igual que la mayoría de los programas de GNU, make acepta una considerable cantidad de opciones de línea de comandos. Las opciones más comunes se listan en la tabla 2.1.

Tabla 2.1. Opciones comunes de línea de comandos de make.

Opción	Propósito
-f nom_archivo	Utiliza el makefile denominado nom_archivo en lugar de uno de los de nombre estándar (GNUmakefile, makefile o Makefile).
-n	Exhibe los comandos que ejecutaría make sin en realidad ejecutarlos. útil para comprobar un makefile.
-I nombre_dir	Añade nombre_dir a la lista de directorios donde buscará make los makefiles incluidos.
-s	Ejecución silenciosa, sin imprimir los comandos que va ejecutando make.
-w	Imprime nombres de directorios cuando make cambia de directorios.
-Wfile	Ejecuta make como si nom_archivo hubiese sido modificado. Igual que -n, muy útil para comprobar makefiles.
-r	Deshabilita todas las reglas integradas en make.
-d	Imprime gran cantidad de información para depuración.
-i	Normalmente, make se detiene si un comando retorna un código de error distinto de cero. Esta opción deshabilita este comportamiento.
-k	Seguir ejecutándose aun si uno de los targets no se lograra construir. Normalmente, si uno de los targets no se lograra construir, make terminaría.
-jN	Correr N comandos en paralelo, donde N es un valor entero pequeño y distinto de cero.

make puede generar mucha salida, de modo que si el lector no se encuentra interesado en analizarla, utilice la opción -s para limitar la salida producida por make.

Las opciones -W y -n le permiten a uno hacer un análisis del tipo “¿qué pasaría si el archivo X cambiase?”

La opción -i existe porque no todos los programas retornan 0 cuando terminan sin inconvenientes. Si uno utilizara un comando así como parte de una regla de un makefile, necesitaría emplear -i a fin de que el proceso de compilación continuara.

La opción -k resulta particularmente útil cuando uno está construyendo un programa por primera vez. Dado que le indica a make que continúe aun si uno o más de los targets no se pudiesen construir, la opción -k le permite a uno ver cuáles targets se construyen sin problemas y cuáles no, permitiéndole concentrar sus esfuerzos de depuración en los archivos con problemas.

Si se está frente a una compilación muy larga, la opción -jN instruye a make de que ejecute N comandos simultáneamente. Aunque esto puede reducir el tiempo total de compilación, también puede imponer una pesada carga sobre el sistema. Esto probablemente resulte tolerable en un sistema

autónomo o en una estación de trabajo individual, pero es totalmente inaceptable en un sistema de producción que requiera rápidos tiempos de respuesta, tal como por ejemplo en un servidor de red o de base de datos.

Ejemplos

Estos ejemplos emplean todos ellos el siguiente makefile, que es el makefile para la mayoría de los programas del capítulo 1. El código fuente asociado también está tomado del capítulo 1. No se preocupe si no comprende algunas de las características de este makefile, porque las mismas se cubren más adelante en este capítulo. Para ver una explicación de cada término, refiérase según el caso a la tabla 1.2 de la página 14, a la tabla 2.2 de la página 46 o a la tabla 2.3 de la página 47. Los comentarios van precedidos en este makefile por el signo numeral (#), tal como se comenta en la página 50. Los signos \$ se utilizan para expandir variables de usuario y predefinidas (ver páginas 42 y 47, respectivamente). Los términos en letras mayúsculas corresponden todos a nombres de variables, en inglés las predefinidas y en español las definidas por usuario. El signo := representa asignación de valores a variables de expansión simple (ver página 43). Para .PHONY ver “Targets ficticios”, página 40. Finalmente, el target .c.o: es una regla implícita, es decir que make la empleará, de ser necesario, esté la misma presente o no en el listado. Las reglas implícitas se comentan en la página 48.

```
#
# Makefile
#
LDFLAGS :=
# Cambie de linea el caracter de comentario que hay en una de
# las dos lineas siguientes segun sea lo que desee optimizar
CFLAGS := -g $(CPPFLAGS)
#CFLAGS := -O2 $(CPPFLAGS)
PROGRAMAS = \
    hola \
    pedant \
    nuevo_hola \
    nuevo_hola-lib \
    no_retorno \
    inline \
    compactar
all: $(PROGRAMAS)
.c.o:
    $(CC) $(CFLAGS) -c $*.c
.SUFFIXES: .c .o
hola: hola.c
pedant: pedant.c
```

```
nuevo_hola: mostrar.c mensajes.c
    $(CC) $(CFLAGS) $^ -o $@

nuevo_hola-lib: mostrar.c libmensajes.a
    $(CC) $(CFLAGS) $< -o $@ -I. -L. -lmensajes

libmensajes.a: mensajes.c
    $(CC) $(CFLAGS) -c $< -o libmensajes.o
    $(AR) rcs libmensajes.a libmensajes.o

no_retorno: no_retorno.c

inline: inline.c
    $(CC) -O2 $< -o $@

compactar: compactar.c

.PHONY : prolijar zip

prolijar:
    $(RM) $(PROGRAMAS) *.o *.a *- *.out
```



```
zip: prolijar
    zip 21510lcd.zip *.c *.h Makefile
```

1. El primer ejemplo ilustra cómo simular una compilación utilizando las opciones `-W` y `-n`.

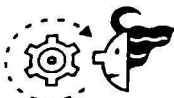


```
$ make -Wmensajes.c -Wmostrar.c -n nuevo_hola-lib
cc -g -c mensajes.c -o libmensajes.o
ar rcs libmensajes.a libmensajes.o
cc -g mostrar.c -o nuevo_hola-lib -I. -L. -lmensajes
```

La opción `-Wnom_archivo` le indica a `make` que actúe como si `nom_archivo` hubiese cambiado, en tanto que `-n` exhibe los comandos que serían ejecutados, pero sin ejecutarlos. Como se puede apreciar a partir de la salida producida, `make` construiría primero el archivo biblioteca, `libmensajes.a`, luego compilaría `mostrar.c` y lo linkearía con la biblioteca para crear `nuevo_hola-lib`. Esta técnica constituye una excelente manera de comprobar las reglas del `makefile`.

**EJEMPLO**

2. Este ejemplo compara la salida normal de `make` con el resultado que se obtiene cuando se emplea la opción `-s` para suprimir dicha salida.

**SALIDA**

```
$ make all
cc -g      hola.c      -o hola
cc -g      pedant.c    -o pedant
cc -g mostrar.c mensajes.c -o nuevo_hola
cc -g -c mensajes.c -o libmensajes.o
ar rcs libmensajes.a libmensajes.o
pedant.c: In function 'main':
pedant.c:8: warning: return type of 'main' is not 'int'
cc -g      no_retorno.c  -o no_retorno
cc -O2 inline.c -o inline
cc -g      compactar.c  -o compactar
cc -g mostrar.c -o nuevo_hola-lib -I. -L. -lmensajes
$ make -s all
pedant.c: In function 'main':
pedant.c:8: warning: return type of 'main' is not 'int'
```

Como el lector puede ver, `-s` suprime toda la salida generada por `make`. El mensaje de advertencia sobre el valor que retorna `main` constituye un diagnóstico generado por `gcc`, el cual `make`, por supuesto, no puede suprimir.

Creación de reglas

Esta sección analiza con mayor detalle la redacción de reglas para `makefile`. En particular, comenta los targets ficticios, las variables de `makefile`, las variables de entorno, las variables predefinidas, las reglas implícitas y las reglas de patrón.

Targets ficticios

Además de los archivos target normales, `make` permite especificar targets *ficticios*. Los targets ficticios se denominan así porque no se corresponden con archivos reales y, al igual que los targets normales, tienen reglas que listan comandos que `make` deberá ejecutar. El último target del `makefile` del ejemplo anterior, `prolijar`, era un target ficticio. Sin embargo, como `prolijar` no exhibía ninguna *dependencia*, sus correspondientes comandos no fueron ejecutados automáticamente.

Este comportamiento de `make` se desprende de la explicación sobre su funcionamiento convencional: cuando encuentra el target `prolijar`, `make` determina si existen o no *dependencias* y, como `prolijar` no tiene dependencias, `make` interpreta que ese target se encuentra actualizado. Para construir dicho target, uno tiene que tipear `make prolijar`. En nuestro ca-

so, *prolijar* eliminaría el archivo ejecutable *editor* y los archivos objeto que le dan origen. Uno podría crear un target de este tipo si se propusiera crear un *tarball* (archivo *tar*) de código fuente, que es el archivo compactado creado utilizando el comando *tar*, o si quisiese comenzar una compilación ya en marcha nuevamente desde cero.

✓ Para obtener una explicación detallada del comando *tar*, ver “Utilización de *tar*”, página 446.

Si efectivamente existiera, sin embargo, un archivo denominado *prolijar*, *make* lo detectaría. Pero, como carecería de *dependencias*, *make* supondría que el mismo se encuentra ya actualizado y no ejecutaría los comandos asociados con el target *prolijar*. Para administrar adecuadamente esta situación, utilice el target especial de *make* *.PHONY*. Todas las *dependencias* del objetivo (*target*) *.PHONY* serán evaluadas como habitualmente, pero *make* no tomará en cuenta la presencia de un archivo cuyo nombre coincide con el de una de las *dependencias* de *.PHONY* y ejecutará de todos modos los comandos correspondientes.



EJEMPLO

Ejemplos

1. Referirse para este ejemplo al *makefile* anterior. Sin el target *.PHONY*, si en el directorio corriente existiera un archivo denominado *prolijar*, el target *prolijar* no funcionará correctamente.



SALIDA

```
$ touch prolijar
$ make prolijar
make: 'prolijar' is up to date.
```

Como se puede apreciar, sin el comando especial *.PHONY*, *make* evaluó las dependencias del target *prolijar*, vio que éste no tenía ninguna y que ya existía un archivo denominado *prolijar* en el directorio corriente, dedujo que el target ya estaba actualizado y no hizo ejecutar los comandos establecidos en la regla.



EJEMPLO

2. Sin embargo, con el comando especial *.PHONY*, el target *prolijar* funciona perfectamente, como se muestra a continuación:



SALIDA

```
$ make prolijar
rm -f hola pedant nuevo_hola nuevo_hola no_retorno inline compactar *.o *.a *-
```

El comando especial *.PHONY* *prolijar* obligó a *make* a pasar por alto la presencia del archivo denominado *prolijar*. En circunstancias ordinarias, lo mejor sería no tener estos archivos de nombres conflictivos en un árbol de directorio de fuente. Como resulta inevitable, sin embargo, que tengan lugar errores y coincidencias, *.PHONY* constituye un excelente resguardo.

Variables

Con el fin de simplificar la edición y mantenimiento del *makefile*, *make* le permite a uno crear y utilizar variables. Una *variable de make* es simplemente un nombre definido en un *makefile* y que representa una cadena de texto; este texto se denomina *valor* de la respectiva variable. *make* puede distinguir entre

cuatro tipos de variables: variables definidas por usuario, variables de entorno, variables automáticas y variables predefinidas. Su empleo es idéntico.

VARIABLES DEFINIDAS POR USUARIO

Las variables de un makefile se definen empleando la forma general:

```
NOMBRE_DE_VARIABLE = valor [...]
```

Por convención, las variables de un makefile se escriben todas en mayúsculas, pero esa no es una condición necesaria. Para obtener el valor de `NOMBRE_DE_VARIABLE`, encierre el nombre entre paréntesis precedido con un signo `$`:

```
$ (NOMBRE_DE_VARIABLE)
```

`NOMBRE_DE_VARIABLE` se expande de modo de hacer disponible el texto almacenado en `valor`, el valor colocado del lado derecho de la sentencia de asignación. Generalmente las variables se definen al comienzo de un makefile.

La utilización de variables en un makefile es antes que nada una conveniencia. Si el valor cambia, uno debe efectuar sólo un cambio en lugar de muchos, simplificándose así el mantenimiento del makefile.

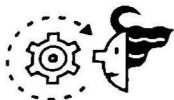


EJEMPLO

Ejemplo

Este makefile muestra cómo funcionan las variables definidas por el usuario. Utiliza el comando `echo` de la interfaz a fin de demostrar cómo se expanden las variables.

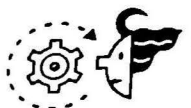
```
PROGRAMAS = prog1 prog2 prog3 prog4
.PHONY : volcar
volcar :
    echo $(PROGRAMAS)
```



SALIDA

```
$ make -s volcar
prog1 prog2 prog3 prog4
```

Como se puede ver, cuando fue empleada con el comando `echo`, `PROGRAMAS` se expandió a sus valores como si fuese un macro. Este ejemplo ilustra también que los comandos presentes en una regla pueden ser cualquier comando válido de interfaz o utilidad o programa de sistema, no solamente invocaciones de compilador. Resulta necesario utilizar la opción `-s` porque `make` muestra en pantalla los comandos a medida que las va ejecutando. En ausencia de `-s`, la salida hubiera sido:



SALIDA

```
$ make volcar
echo prog1 prog2 prog3 prog4
prog1 prog2 prog3 prog4
```


EXPANSIÓN RECURSIVA CONTRA EXPANSIÓN SIMPLE DE VARIABLES

make en realidad emplea dos tipos de variables, recursivamente expandidas y simplemente expandidas. Las *variables recursivamente expandidas* son expandidas tal cual cuando son referenciadas, o sea, si la expansión contuviese otra referencia a una variable, esta última será asimismo expandida. La expansión continúa hasta que no queden más variables por expandir, a lo que se debe el nombre de esta modalidad, expansión recursiva. Por ejemplo, consideremos las variables `DIR_SUPERIOR` y `DIR_FUENTE`, definidas como sigue:

```
DIR_SUPERIOR = /home/kurt_wall/mi_proyecto
```

```
DIR_FUENTE = $(DIR_SUPERIOR)/fuente
```

Así, `DIR_FUENTE` tendrá el valor `/home/kurt_wall/mi_proyecto/fuente`. Esto funciona como se lo esperaba y deseaba. Sin embargo, uno no puede agregar texto al final de una variable definida con anterioridad. Consideremos la definición de la siguiente variable:

```
CC = gcc
```

```
CC = $(CC) -g
```

Claramente, lo que en definitiva se desea es obtener `CC = gcc -g`. Esto no es lo que se obtendrá, sin embargo. Cada vez que se la referencia `$(CC)` es expandida recursivamente, de modo que uno termina con un lazo infinito: `$(CC)` insistirá en expandirse a `$(CC)`, y nunca se llegará a la opción `-g`. Afortunadamente, make detecta esto e informa un error:

```
*** Recursive variable 'CC' references itself (eventually). Stop.
```

La figura 2-1 ilustra el problema presente con la expansión recursiva de variables.

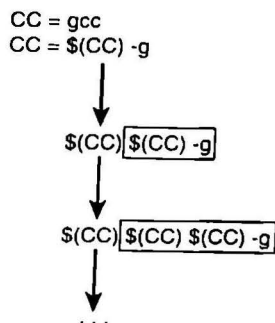


Figura 2.1. Los peligros de las variables de make expandidas recursivamente.

Como se puede advertir en la figura, cada vez que se trata de agregar algo a una variable previamente definida que se expanda recursivamente se da comienzo a un lazo infinito. El texto recuadrado representa el resultado de la expansión previa, mientras que el elemento recursivo se representa por el agregado de `$(CC)` al comienzo de esa expansión. Como se ve con claridad, la variable `CC` nunca resultará evaluada.

Para evitar este problema, `make` también utiliza *variables de expansión simple*. En lugar de ser expandidas cuando se las referencia, las variables de *expansión simple* son examinadas una vez cuando se las define; todas las referencias internas a variables son procesadas inmediatamente. La sintaxis de la definición es ligeramente diferente:

```
CC := gcc
CC += -g
```

La primera definición utiliza `:=` para hacer `CC` igual a `gcc` y la segunda emplea `+=` para añadir `-g` a la primera definición, de modo que el valor final `CC` sea `gcc -g` (`+=` opera en `make` de la misma manera en que lo hace en C). La figura 2-2 ilustra gráficamente cómo funcionan las variables de expansión simple. Si se presentan problemas cuando utiliza variables de `make` o aparece el mensaje de error “`NOMBRE_DE_VARIABLE` se referencia a si misma”, es tiempo de comenzar a utilizar variables de expansión simple. Algunos programadores emplean sólo variables de expansión simple con el fin de evitar problemas no previstos. ¡Como esto es Linux, no le costará nada elegir por sí mismo!

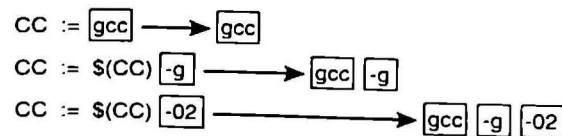


Figura 2.2. Las variables de expansión simple son procesadas completamente la primera vez que son examinadas.

La figura 2-2 muestra que cada vez que se referencia por primera vez una variable de expansión simple, ésta resulta totalmente expandida. Las asignaciones de variables del lado izquierdo del signo igual adoptarán los valores que se encuentran del lado derecho, y dichos valores no contendrán referencias a otras variables.

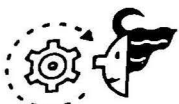


EJEMPLO

Ejemplos

1. Este makefile utiliza variables expandidas recursivamente. Como se hace referencia repetidamente a la variable `CC`, el `make` no funcionará. El archivo `hola.c` está tomado del capítulo 1.

```
CC = gcc
CC = $(CC) -g
hola: hola.c
    $(CC) hola.c -o hola
```



SALIDA

```
$ make hola
```

```
Makefile:5: *** Recursive variable 'CC' references itself (eventually). Stop.
```

Como sería de esperar, `make` detectó el empleo recursivo de `CC` y se detuvo.

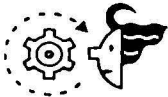
**EJEMPLO**

2. Este makefile utiliza variables de expansión simple en lugar de las variables de expansión recursiva del primer ejemplo.

```
CC := gcc
CC += -g
```

```
hola: hola.c
```

```
    $(CC) hola.c -o hola
```



```
$ make hola
```

```
gcc -c hola.c -o hola
```

SALIDA

Esta vez, el programa compila normalmente.

VARIABLES DE ENTORNO

Las *variables de entorno* son las versiones de make de todas las variables de entorno de la interfaz. Cuando se inicia, make lee todas las variables definidas en el entorno de la interfaz y crea variables con los mismos nombres y valores. Sin embargo, las variables del mismo nombre presentes en el makefile invalidan las variables de entorno, de modo que tenga cuidado.

**EJEMPLO**

Ejemplo

Este makefile utiliza la variable \$HOME que hereda de la interfaz como parte de una regla para construir el target foo.

```
foo : $(HOME)/foo.c
```

```
    gcc $(HOME)/foo.c -o $(HOME)/foo
```



```
$ make all
```

```
make: *** No rule to make target '/home/kurt_wall/foo.c', needed by 'foo'. Stop.
```

SALIDA

El mensaje de error exhibido es un tanto engañoso. Cuando make no pudo encontrar /home/kurt_wall/foo.c, listado como dependencia del target foo, interpretó que debía de alguna manera construir primero foo.c. Sin embargo, no pudo encontrar una regla para ello, de modo que exhibió el mensaje de error y terminó. Lo que se pretendía, sin embargo, era demostrar que make heredó la variable \$HOME del entorno de la interfaz (/home/kurt_wall, en este caso).

VARIABLES AUTOMÁTICAS

Las *variables automáticas* son variables cuyos valores son evaluados cada vez que se ejecuta una regla, basándose en el target y en las *dependencias* de esa regla. Las variables automáticas se emplean para crear *reglas patrón*. Las reglas patrón son instrucciones genéricas, tales como una regla que indique cómo compilar un archivo .c arbitrario a su correspondiente

archivo .o. Las variables automáticas tienen un aspecto bastante críptico. La tabla 2.2 contiene una lista parcial de variables automáticas.

✓ Para ver con mayor detalle todo lo referido a reglas patrón, ver “Reglas patrón”, página 49.

Tabla 2.2. Variables automáticas

Variable	Descripción
<code>\$@</code>	Representa el nombre de un archivo target presente en una regla
<code>\$*</code>	Representa la porción básica (o raíz) de un nombre de archivo
<code>\$<</code>	Representa el nombre de archivo de la primera <i>dependencia</i> de una regla
<code>^</code>	Se expande para dar lugar a una lista delimitada por espacios de todas las <i>dependencias</i> de una regla
<code>?</code>	Se expande para dar lugar a una lista delimitada por espacios de todas las dependencias de una regla que sean más nuevas que el target
<code>\$(@D)</code>	Si el target designado se encuentra en un subdirectorio, esta variable representa la parte correspondiente a los directorios de la ruta de acceso al mismo
<code>\$(@F)</code>	Si el target designado se encuentra en un subdirectorio, esta variable representa la parte correspondiente al nombre del archivo de la ruta de acceso al mismo

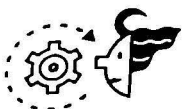


EJEMPLO

Ejemplo

El fragmento de makefile que viene a continuación (del makefile del capítulo 4, “Procesos”) utiliza las variables automáticas `$*`, `$<`, y `$@`.

```
CFLAGS := -g
CC := gcc
.c.o:
    $(CC) $(CFLAGS) -o $*.c
imprpids: imprpids.c
    $(CC) $(CFLAGS) $< -o $@
ids: ids.c
    $(CC) $(CFLAGS) $< -o $@
$ make imprpids ids
gcc -g imprpids.c -o imprpids
gcc -g ids.c -o ids
```



SALIDA

El primer target es una regla implícita o predefinida. (Las reglas implícitas se explican en detalle más adelante en este mismo capítulo.). Establece que, para cada nombre de archivo que tenga la extensión `.c`, se creará un archi-

vo objeto de extensión .o, utilizando el comando `$(CC) $(CFLAGS) -c`. El término `$*.c` representa cualquier archivo del directorio corriente que tenga la extensión .c; en este caso, `prpids.c` y `ids.c`.

Como lo mostraron los comandos volcados al dispositivo estándar de salida (que, salvo advertencia en contrario, consideraremos siempre la pantalla del monitor), `make` reemplazó `$<` con la primera *dependencia* presente en cada regla, `prpids.c` y `ids.c`. De manera similar, la variable automática `$@` presente en los comandos destinados a construir `prpids` y `ids` es reemplazada por los nombres de los targets para estas reglas, `prpids` y `ids`, respectivamente.

VARIABLES PREDEFINIDAS

Además de las variables automáticas listadas en la tabla 2.2, el `make` de GNU pre-define cierto número de otras variables que son utilizadas ya sea como nombres de programas o para transferir indicadores y argumentos a esos programas. La tabla 2.3 lista las variables predefinidas de `make` de empleo más frecuente. El lector ya ha visto algunas de ellas en los `makefiles` de ejemplo de este capítulo.

Tabla 2.3. Variables predefinidas para nombres de programa e indicadores

Variable	Descripción	Valor predeterminado
AR	Nombre de archivo del programa de mantenimiento de archivos compactados	ar
AS	Nombre de archivo del ensamblador	as
CC	Nombre de archivo del compilador de C	cc
CPP	Nombre de archivo del preprocesador de C	cpp
RM	Programa para eliminar archivos	rm -f
ARFLAGS	Indicadores para el programa de mantenimiento de archivos compactados	rv
ASFLAGS	Especificadores de opciones para el ensamblador	No hay
CFLAGS	Especificadores de opciones para el compilador de C	No hay
CPPFLAGS	Especificadores de opciones para el preprocesador de C	No hay
LDFLAGS	Especificadores de opciones para el linker	No hay

El último ejemplo de un `makefile` utilizó varias de estas variables predefinidas. El lector puede redefinir estas variables en su `makefile`, aunque en la mayoría de los casos sus valores predeterminados son razonables. Redefínalas cuando el comportamiento predeterminado no satisfaga sus necesidades, o cuando sepa que un programa presente en un sistema determinado no tiene la misma sintaxis para las llamadas.



EJEMPLO

Ejemplo

El `makefile` de este ejemplo rescribe la del ejemplo anterior de modo de utilizar solamente variables automáticas y predefinidas donde sea posible, y no redefina los valores predeterminados de las variables predefinidas que provee `make`.

```
prpids: prpids.c
        $(CC) $(CFLAGS) $< -o $@ $(LDFLAGS)

ids: ids.c
        $(CC) $(CFLAGS) $< -o $@ $(LDFLAGS)
```

La salida que genera este makefile es levemente diferente de la del ejemplo anterior:



```
$ make
cc prpids.c -o prpids
cc ids.c -o ids
```

Esta salida muestra que make utilizó el valor predeterminado de CC, cc, en lugar de gcc. En la mayoría de los sistemas de Linux, sin embargo, cc constituye un vínculo simbólico con gcc (o egcs), de modo que la compilación funcionó correctamente.

Reglas implícitas

Hasta ahora, los makefiles de este capítulo han utilizado reglas explícitas, reglas que redacta el propio programador. make incluye un extenso conjunto de reglas implícitas, o predefinidas, también. Muchas de ellas son para propósitos especiales y tienen un empleo limitado, de modo que este capítulo cubre sólo unas pocas de las reglas implícitas más comúnmente utilizadas. Las reglas implícitas simplifican el mantenimiento de los makefiles.

Supongamos que un makefile tuviese estas dos reglas:

```
prog : prog.o
```

```
prog.o : prog.c
```

las dos reglas listan *dependencias*, pero no reglas para construir sus targets. En lugar de terminar su ejecución, sin embargo, make intentará encontrar y utilizar reglas implícitas que le permitan construir los targets (el lector puede verificar que make busca reglas implícitas observando la salida generada por el depurador cuando se utiliza la opción -d).

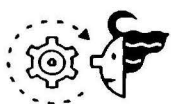


Ejemplo

El makefile de aspecto sumamente breve que viene a continuación creará prpids utilizando dos de las reglas implícitas de make. La primera define cómo crear un archivo objeto a partir de un archivo de código fuente de C. La segunda define cómo crear un archivo binario (o ejecutable) a partir de un archivo objeto.

```
prpids : prpids.o
```

```
prpids.o : prpids.c
```

SALIDA

S make

cc -c prpids.c -o prpids.o

cc prpids.o -o prpids

make invoca dos reglas implícitas para construir `imprpids`. La primera regla establece, en esencia, que para cada archivo objeto `este_archivo.o`, se deberá buscar el correspondiente archivo fuente `este_archivo.c` y construir el archivo objeto con el comando `cc -c este_archivo.c -o este_archivo.o`. De modo que make buscó un archivo fuente en C denominado `imprpids.c` y lo compiló para obtener el archivo objeto `imprpids.o`. Para construir el target predeterminado, `imprpids`, make utilizó otra regla implícita que establece para cada archivo objeto cuyo nombre sea `este_archivo.o`, efectuar el *linkeo* necesario para obtener el archivo ejecutable final utilizando el comando `cc este_archivo.o -o este_archivo`.

Reglas patrón

Las reglas patrón proveen una manera de que uno defina sus propias reglas implícitas. Las reglas patrón tienen el aspecto de reglas normales, excepto que el target contiene exactamente un carácter (%) que representa cualquier cadena no vacía. Las *dependencias* de una regla de este tipo también emplean % con el fin de coincidir con el target. Así, por ejemplo, la regla:

```
%o : %.c
```

Le indica make que construya cualquier archivo objeto `este_archivo.o` a partir de un archivo fuente correspondiente denominado `este_archivo.c`. Esta regla patrón también resulta ser una de las reglas patrón predefinidas de make pero, igual que lo que sucede con las variables predefinidas, uno puede redefinirlas para acomodar sus propias necesidades.

```
%o : %.c
```

```
$(CC) -c $(CFLAGS) $(CPPFLAGS) $< -o $@
```



EJEMPLO

Ejemplo

El makefile que viene a continuación es similar al anterior, excepto que utiliza una regla patrón para crear una regla implícita personalizada, definida por el usuario, para compilar y *linkear* código fuente en C, y emplea una regla implícita predefinida para *linkear* el archivo objeto a fin de crear el archivo binario final.

```
CFLAGS := -g -O3 -c
```

```
CC := gcc
```

```
#
```

```
# Redefine la regla patron predeterminada
```

```
# %.o : %.c
```

```
#
```

```

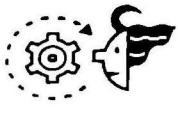
%.o : %.c
    $(CC) $(CFLAGS) $< -o $@

#
# Este comentario va aqui solo por colocar un comentario
#

imprpids : imprpids.o

imprpids.o : imprpids.c
$ make
gcc -g -O3 -c imprpids.c -o imprpids.o
gcc  imprpids.o  -o imprpids

```

**SALIDA**

La regla patrón definida por el usuario efectuó varias cosas:

- Cambió CC a gcc.
- Añadió el especificador de depuración de gcc 's, -g, y el especificador de optimización, -O3, a CFLAGS.
- Utilizó las variables automáticas \$< y \$@ para reemplazar los nombres sucesivos de la primera dependencia y el target cada vez que la regla fue aplicada.

Como se puede ver a partir de la salida de make, la regla personalizada fue aplicada al makefile tal como se la especificó, dando origen a un archivo ejecutable sumamente optimizado que contiene, además, información integrada en el mismo sobre su depuración.

Comentarios

Se pueden insertar comentarios en un makefile precediendo el comentario con el signo de numeral (#). Cuando make encuentra un comentario, no procesa ni el signo de numeral ni el resto de la línea donde éste se encuentra. En un makefile los comentarios pueden ir colocados en cualquier lado. Se debe otorgar especial consideración a los comentarios que aparecen en los comandos, porque la mayoría de las interfaces tratan a # como metacarácter (generalmente como un delimitador de comentarios). Por lo que concierne a make, una línea que contiene sólo un comentario equivale, para todo propósito práctico, a una línea en blanco. El ejemplo anterior ilustró el empleo de comentarios. make ignoró totalmente las cuatro líneas que consistían sólo el delimitador de comentarios y las dos líneas que contenían sólo texto y no comandos.

Targets útiles para un makefile

Además del target `prolijar` ilustrado anteriormente en este capítulo, en los makefiles habitan generalmente varios otros targets. Un target denominado `install` desplaza el archivo final ejecutable, cualquier biblioteca o *script* de interfaz requeridos y la documentación que pudiese existir a sus

alojamientos finales en el *filesystem* y establece adecuadamente permisos de archivos y de propiedad.

El target `instalar` también típicamente compila el programa y puede, además, correr una sencilla comprobación para verificar que el programa se haya compilado correctamente. Un target `uninstall` suprime los archivos instalados por el target `install`.

Un target `dist` es una manera conveniente de preparar un paquete de distribución. Como mínimo absoluto, el target `dist` elimina archivos antiguos, tanto ejecutables como objeto, del directorio donde se llevó a cabo la construcción, y crea un archivo comprimido, tal como un archivo `tar` o `tarball` realizado con `gzip`, listo para ser colocado en páginas de la World Wide Web y en sitios FTP.

NOTA

El programa `gzip` es un compresor y descompresor multipropósito de archivos que es compatible con la utilidad clásica de compresión de UNIX. Es uno de los programas más populares de los proyectos GNU y se encuentra disponible para prácticamente todos los sistemas operativos en uso.



EJEMPLO

Ejemplo

El siguiente `makefile` ilustra cómo crear los targets `install`, `dist`, y `uninstall`, utilizando el ya sumamente trillado programa `hola.c`.

```
hola : hola.c

install : hola
    install $< $(HOME)

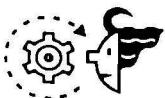
.PHONY : dist uninstall

dist :
    $(RM) hola *.o core
    tar czvf hola.tar.gz hola.c Makefile
```

```
desinstalar :
    $(RM) $(HOME)/hola

$ make install
cc      hola.c      -o hola
install hola /dir_principal/kurt_wall

$ make dist
rm -f hola *.o core
tar czvf hola.tar.gz hola.c Makefile
hola.c
```



SALIDA

```

Makefile
S make uninstall
rm -f /dir_principal/kurt_wall/hola

```

`make install` primeramente compila `hola` y lo instala en el directorio especificado. Esto ilustra una de las ventajas de utilizar variables de `make`: `$HOME` evalúa al valor correcto en cualquier sistema en que corra `make`, y por lo tanto no requiere ser especificada explícitamente en el `makefile`. `make dist` elimina todos los archivos residuales que pueda haber dejado la construcción, tales como módulos objeto y otros archivos temporarios, y crea lo que equivale a una distribución de código fuente lista para ser publicada en Internet. El target `uninstall`, finalmente, elimina en segundo plano el programa instalado.

Administración de errores

Si el lector llegase a tener problemas cuando utiliza `make`, la opción `-d` le indica a `make` que imprima gran cantidad de información adicional para depuración, además de mostrar los comandos que se encuentre ejecutando. La salida obtenida mediante esta opción puede ser abrumadora porque la información volcada mostrará qué es lo que hace `make` internamente y por qué. La salida generada por la opción de depuración incluye la siguiente información:

- Qué archivos evalúa `make` para efectuar una reconstrucción.
- Qué archivos están siendo comparados y cuáles son los resultados de la comparación.
- Qué archivos son los que verdaderamente necesitan ser reconstruidos.
- Qué reglas implícitas considera `make` que va a utilizar.
- Qué reglas implícitas decide utilizar `make`, y los correspondientes comandos que ejecutará.

La siguiente lista contiene los mensajes de error más comunes que pueden ser encontrados cuando se utiliza `make` y sugiere cómo resolverlos. Para obtener la documentación completa, referirse al manual de `make` o, mejor aún, a las páginas de información de `make` (utilice el comando `info "GNU make"`).

- No rule to make target 'target'. Stop - `make` no pudo encontrar una regla adecuada en el `makefile` que le permitiera construir el target designado y no puede hallar reglas predeterminadas que le sean de utilidad. La solución es ubicar el target causante del problema y añadir una regla que permita crearlo, o modificar la regla existente.
- 'target' is up to date. Las *dependencias* para el *target* designado no han cambiado (son más viejas que el target). Esto no es realmente un mensaje de error, pero, si el lector quisiera forzar la reconstrucción del target, sencillamente usa la utilidad `touch` para modificar la fecha y hora del archivo. Esto logrará que `make` reconstruya el target en cuestión.

- Target 'target' not remade because of errors. Tuvo lugar un error mientras se construía el target designado. Este mensaje sólo aparece cuando se utiliza la opción `-k` de `make`, que obliga a éste a continuar aunque ocurran errores (ver tabla 2.1, página 37). Cuando aparece este error existen diversas soluciones posibles. El primer paso sería tratar de construir sólo el target en cuestión y, basándose en los errores generados, determinar el siguiente paso apropiado.

✓ La opción `-k` de `make` se halla cubierta en mayor detalle en la tabla 2.1, “Opciones comunes de línea de comandos de `make`”, página 37.

- `nom_programa`: Command not found-`make` no pudo encontrar `nom_programa`. Esto ocurre habitualmente porque `nom_programa` ha sido mal tipeado o no se encuentra incluido en la variable de entorno `$PATH`. Indistintamente utilice el nombre completo del programa, incluyendo su ruta de acceso, añada la ruta completa de acceso a `nom_programa` a la variable `$PATH` de `makefile`.
- Illegal option - option. La invocación de `make` incluía una opción que `make` no reconoció. No utilice la opción que ocasionó el conflicto o verifique su sintaxis.

Lo que viene

Este capítulo le presentó al lector el comando `make`, explicándole por qué resulta útil y cómo utilizarlo. El lector cuenta ahora con la suficiente base como para comenzar a escribir y compilar programas simples en el entorno de desarrollo de Linux. Luego de presentarle una vista preliminar del programa que habrá construido cuando haya completado este libro (capítulo 3, “Acerca del proyecto”), la Parte II le permite comenzar a programar concretamente al enseñarle cómo programar Linux a nivel de sistema. El lector comenzará con el modelo de proceso de Linux, cubierto en el capítulo 4, “Procesos.”